

南開大學
Nankai University

南　　开　　大　　学

计　算　机　学　院

编译原理上机实验报告

预备工作：了解你的编译器、LLVM IR 与汇编编程

姓名：张龙飞、袁煜杰

学号：2413625、2412303

专业：计算机科学与技术

2026 年 10 月 11 日

摘要

本报告记录两人的预备实验。袁煜杰完成六组手写 LLVM IR，与参考程序、本机和 RISC-V 执行结果比较，236 项检查全部通过；MLIR 进阶部分保存 Linalg 到 LLVM 的逐层输出，并验证 VecAdd 的 16 个元素。张龙飞已有编译流程与循环阶乘汇编实验，相关分析和更多手写汇编仍待其完善。本版本为共同报告整合稿。

关键字：编译器；LLVM IR；RISC-V 汇编；SysY；链接

目录

一、 引言 **1**

（一） 实验背景与目的 1

（二） 实验环境（张龙飞） 1

（三） 实验环境（袁煜杰） 1

（四） 小组分工 1

二、 编译系统工作流程研究（张龙飞） **2**

（一） 实验方案 2

（二） 预处理器 2

（三） 编译器 3

2.3.1 编译前端：生成 LLVM IR 3

2.3.2 编译后端：生成目标汇编 4

（四） 汇编器 4

（五） 链接器 5

（六） 运行验证 5

（七） 进一步探索：优化选项的影响 5

三、 LLVM IR 与汇编编程实践 **6**

（一） 任务范围与实现方法 6

（二） 实验环境与复现流程 6

（三） 示例设计与特性覆盖 6

（四） 短路控制流与可观测副作用 7

（五） 循环、函数与数据表示 7

（六） mem2reg 观察与结果验证 8

（七） 结论与搭档交接 8

（八） 手写 RISC-V 汇编程序及验证 8

（九） 共同验收状态 9

四、 进阶要求：MLIR 多级中间表示探索 **9**

（一） 探索目标与证据边界 9

（二） AscendNPU IR VecAdd 的方言与层级 9

（三） 可执行的上游 VecAdd 设计 10

（四） 逐层 lowering 与检查点 10

（五） 末端验证与分析 11

五、 总结与体会 **11**

一、 引言

（一） 实验背景与目的

SysY 是全国大学生编译系统设计赛定义的简化 C 语言。本课程上机大作业要求实现一个 SysY 编译器（词法分析、语法分析、类型检查与中间代码生成、代码优化、目标代码生成），目标平台为 RISC-V。本次预备工作是上机大作业的第一个模块，目的在于：

1. 以 GCC / LLVM(Clang) 为研究对象，深入探究语言处理系统的完整工作过程，理解预处理器、编译器（前端与后端各阶段）、汇编器、链接器的功能；
2. 熟悉 LLVM IR 中间语言与 RISC-V 汇编语言，设计覆盖 SysY 语言特性的示例程序，手写等价的 LLVM IR 与 RISC-V 汇编程序，链接 SysY 运行时库并验证运行结果的正确性；
3. 为后续词法分析器、语法分析器、中间代码生成与目标代码生成等模块的实现建立直观认识与技术储备。

（二） 实验环境（张龙飞）

本实验在 WSL2（Windows Subsystem for Linux）中进行，使用 QEMU 用户态模拟器运行 RISC-V 目标程序。主要工具及版本如表 1 所示。

项目	版本 / 说明
操作系统	WSL2：Ubuntu 22.04.5 LTS（x86_64）
GCC / G++	11.4.0
Clang / LLVM 工具链	14.0.0（含 llvm-as、lli、llc、opt、llvm-dis）
Flex / Bison	2.6.4 / 3.8.2
RISC-V 交叉工具链	riscv64-linux-gnu-gcc / g++ 11.4.0
模拟器	QEMU 7.0.0（qemu-riscv64 用户态模式，/opt/qemu 源码编译）

表 1: 实验环境与工具版本

（三） 实验环境（袁煜杰）

实际运行环境为 WSL Ubuntu 26.04.1 LTS（x86_64），Clang/LLVM 21.1.8，RISC-V GCC 15.2.0，QEMU 10.2.1。MLIR 21.1.8 工具从 Ubuntu 官方源下载并在项目内解压。详细命令和版本保存在实验结果目录中。

（四） 小组分工

袁煜杰（ashalghy-hub）负责基本要求二的 LLVM IR 手写程序与验证、进阶 MLIR 探索及相应报告；张龙飞（3769lkj）负责基本要求二的手写 RISC-V 汇编，并继续完善已有工作流程实验分析。两人共同整合一份报告，开头说明分工，分别提交。本版本保留搭档尚未补齐的内容，完整的袁煜杰部分另提供独立附件。

二、 编译系统工作流程研究（张龙飞）

本小题以 PPT 示例的 C++ 版阶乘程序 `fact.cpp` 为研究载体，利用编译器的命令行选项逐步获得各阶段输出（实验方案见表 2），研究其与源程序的关系。

（一） 实验方案

研究载体：阶乘程序 fact.cpp（PPT 示例）

```
#include <iostream>
using namespace std;
int main()
{
    int i, n, f;

    cin >> n;
    i = 2;
    f = 1;
    while (i <= n)
    {
        f = f * i;
        i = i + 1;
    }
    cout << f << endl;
}
```

阶段	命令选项	产物
预处理	`clang++ -E`	`fact.i`
编译（前端）	`clang++ -emit-llvm -S`	`fact.ll`（LLVM IR）
编译（后端）	`g++ -S` / `riscv64-linux-gnu-g++ -S`	`fact.s`（汇编）
汇编	`clang++ -c`	`fact.o`（目标文件）
链接	`g++` / `riscv64-linux-gnu-g++ -static`	可执行程序
运行验证	`lli` / `qemu-riscv64`	运行结果

表 2: 各编译阶段对应的命令选项与产物

（二） 预处理器

源程序仅 17 行，经 `clang++ -E` 预处理后得到 30194 行的 `fact.i`——C++ 标准库头文件 `<iostream>` 展开了数万行声明。

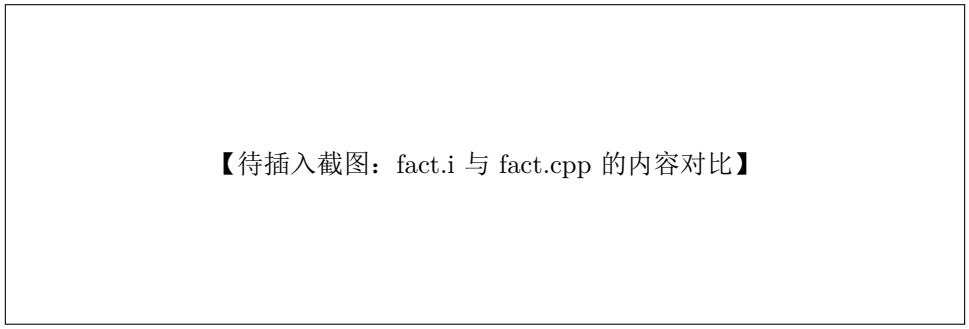


图 1: 预处理输出与源程序的对比

【待补全：预处理前后对比分析，例如 `#include` 与 `#define` 的处理、注释的去留、可用 `-D` 选项自定义宏做进一步观察，以及 C++ 头文件展开规模远大于 C 头文件的原因】

（三） 编译器

2.3.1 编译前端：生成 LLVM IR

`clang++ -emit-llvm -S` 生成 111 行 LLVM IR（`fact.ll`），关键片段如下：

fact.ll 关键片段

```
@_ZSt3cin  = external global %"class.std::basic_istream", align 8
@_ZSt4cout = external global %"class.std::basic_ostream", align 8

define dso_local noundef i32 @main() #4 {
  %1 = alloca i32, align 4      ; return value
  %2 = alloca i32, align 4      ; i
  %3 = alloca i32, align 4      ; n
  %4 = alloca i32, align 4      ; f
  store i32 0, i32* %1, align 4
  %5 = call ... @_ZNSirsERi(@_ZSt3cin, i32* %3)      ; cin >> n
  store i32 2, i32* %2, align 4                      ; i = 2
  store i32 1, i32* %4, align 4                      ; f = 1
  br label %6
6:                                                   ; while (i <= n)
  %7 = load i32, i32* %2, align 4
  %8 = load i32, i32* %3, align 4
  %9 = icmp sle i32 %7, %8
  br i1 %9, label %10, label %16
10:
  %11 = load i32, i32* %4
  %12 = load i32, i32* %2
  %13 = mul nsw i32 %11, %12                         ; f = f * i
  ...
16:
  %17 = load i32, i32* %4, align 4
  %18 = call ... @_ZNSolsEi(@_ZSt4cout, i32 %17)     ; cout << f
  %19 = call ... @_ZNSolsEPFRSoS_E(%18, ... endl)    ; << endl
  ret i32 %20
```

```
}
```



图 2: LLVM IR 与控制流结构

【待补全：IR 与源程序的对应关系分析】

2.3.2 编译后端：生成目标汇编

分别用 `g++ -S`（x86_64，AT&T 风格）与 `riscv64-linux-gnu-g++ -S`（RISC-V）生成汇编，对比两者风格。



图 3: 同一代码在 x86 与 RISC-V 下的汇编对比

【待补全：x86 与 RISC-V 汇编的对比分析】

（四） 汇编器

`clang++ -c` 将汇编翻译为目标文件 `fact.o`，可用 `objdump -d` 反汇编查看机器码，`objdump -r` 查看重定位项。

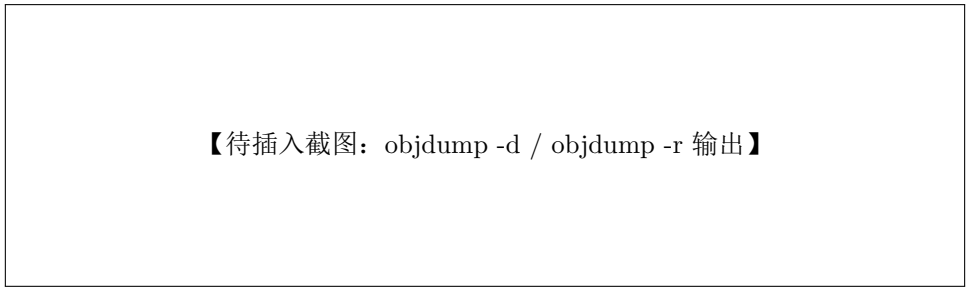


图 4: 目标文件的反汇编与重定位信息

【待补全：汇编器功能分析】

（五） 链接器

链接阶段将目标文件与 C++ 运行库（`std::cin`/`std::cout` 所在的 libstdc++）组装为可执行程序。动态链接与静态链接产物对比如图 5 所示。

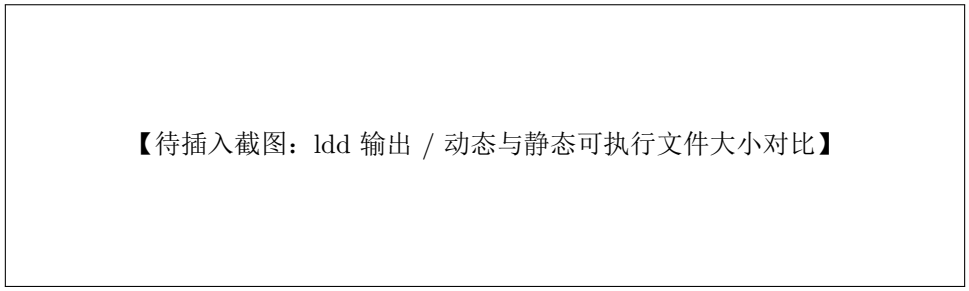


图 5: 动态链接与静态链接对比

【待补全：链接器功能分析】

（六） 运行验证

四种方式运行同一程序，结果一致（输入 5，输出 120），说明各阶段产物等价：

运行验证

```
$ echo 5 | ./fact_x86                # x86 可执行（g++ 链接产物）
120
$ echo 5 | lli fact.ll               # LLVM IR 解释执行
120
$ echo 5 | ./fact_fromll_x86         # 由 IR 直接重编译的可执行
120
$ echo 5 | qemu-riscv64 ./fact_rv64  # RISC-V 可执行（QEMU 模拟）
120
```

（七） 进一步探索：优化选项的影响

调整优化级别（`-O0/-O1/-O2/-Os`），观察 LLVM IR 与 RISC-V 汇编的变化，结果如表 3 所示。

程序 / 产物	-O0	-O1	-O2	-Os
fact：LLVM IR 行数	111	163	250	152
fact：RISC-V 汇编行数	156	97	146	89
fib：LLVM IR 行数	124	248	247	153
fib：RISC-V 汇编行数	177	156	231	118

表 3: 不同优化级别下 LLVM IR 与 RISC-V 汇编的规模对比

【待补全：结合表 3 及更多探索（如 `-g` 选项、微调源程序）分析优化器行为】

三、 LLVM IR 与汇编编程实践

（一） 任务范围与实现方法

本部分由袁煜杰完成，对应预备工作的基本要求二中的 LLVM IR 编程任务。在已有循环阶乘入门例子之外，设计六组 SysY 源程序，并手写对应的 LLVM IR。原始源文件位于 `prep/llvm_ir/`，每组包含同名的 `.sy` 和 `.ll`。优化 IR 与自动生成的 RISC-V 汇编单独放在 `results/`，避免与手写文件混淆。

实现遵循课程提供的 SysY 定义及文法补充：整数采用 i32，浮点采用 float，普通表达式中的逻辑非结果需转换为整数。LLVM IR 的局部寄存器值遵循 SSA，而保存在内存中的变量还需要 load/store；有这些访存操作并不意味着整个 LLVM IR 是非 SSA。控制流中的合流值通过 phi 表示，具体语义参见 LLVM 语言参考手册 [1]。

（二） 实验环境与复现流程

本次实际环境为 Ubuntu WSL、x86_64，Ubuntu 26.04.1 LTS，Clang/LLVM 21.1.8、RISC-V Linux GCC 15.2.0、QEMU 10.2.1。此环境与搭档已有的 Clang 14 环境分别记录，不能混用版本或编译产物来源。RISC-V 目标使用 RV64GC、LP64D 调用约定与静态链接。

本实验链接已有最小教学运行库 `prep/manual/sylib.c` 中的 getint、putint 和 putch。该实现足以支撑当前例子，但尚未取得课程正式运行库，因此未声称完成正式运行时的全部接口或 ABI 验收。正式库取得后需按同一流程复测。`.sy` 文件不包含 C 头文件；参考编译通过命令行强制包含库声明，且所选示例均避开 SysY 与 C 语义不同的语法。

```
python3 prep/llvm_ir/verify.py
llvm-as sample.ll -o sample.bc
opt -passes=verify -disable-output sample.ll
opt -S '-passes=default<O2>' sample.ll -o sample_O2.ll
llc -mtriple=riscv64-linux-gnu -mattr=+m,+a,+f,+d,+c \
    -target-abi=lp64d sample.ll -o sample_generated_rv.s
```

上面的 sample 命令说明单组操作，实际完整命令由验证脚本记录。脚本每次重新构建程序，检查编译退出码，对运行设置超时，逐项比较标准输出及退出码；不依赖已有可执行文件。二进制写入 build，命令与诊断写入 `commands.json`，输入和预期/实际输出写入 `latest.json`。

（三） 示例设计与特性覆盖

示例	主要覆盖	输入数
arithmetic	加减乘除与余数、单目正负、常量、块遮蔽、逻辑非转整数	6
short_circuit	if/else、逻辑与或短路、全局副作用、自定义函数	6
loop_control	while、continue、break、合流与多条回边	6
functions	带参函数、递归、整数返回、void 函数及返回	5
array_float	二维数组、部分初始化、数组形参、浮点运算与转换	5
promote_demo	内存形式循环及 mem2reg 对照	5

表 4: 本次手写 LLVM IR 的实验设计

算术例子输入 n，计算 $(n+3)\times 2-n/2+n\%3+7$，并分别输出 $-n$、$+n$、n+!n。内层同名变量固定为 7，外层 n 在后续输出中仍保持原值，用于验证遮蔽而非覆盖。负数整除按向零截断计算，余数为 $n-\mathrm{trunc}(n/3)\times 3$；不能直接用 Python 向下取整除法作为预期。逻辑非先由 icmp 产生 i1，再由 zext 转为 i32 后参与加法。

（四） 短路控制流与可观测副作用

短路例子包含 `n != 0 && 12 / n > 2` 和 `n == 0 || touch()`。touch 每次调用将全局 hits 加一。以 n=0 为例，正确程序既不除以零，也不调用 touch，最终 hits=0；非零输入则调用 touch 一次。这比只比较纯表达式的真假更能检验执行顺序。

```
%nonzero = icmp ne i32 %n, 0
br i1 %nonzero, label %and.rhs, label %and.false
and.rhs:
  %q = sdiv i32 12, %n
  %large = icmp sgt i32 %q, 2
  br i1 %large, label %and.true, label %and.false
and.join:
  %and.value = phi i32 [1, %and.true], [0, %and.false]
```

代码为关键片段，省略的基本块和声明见同名完整源文件。右操作数置于独立块中，而不是先执行两边再用 and/or 合并；后者会错误执行除法或副作用。测试 n=-3、0、1、4、5、12，覆盖受保护分母、条件真/假和副作用次数。

（五） 循环、函数与数据表示

循环例子逐个处理 1 到 n，跳过 2，在数值超过 5 时退出。continue 的边直接回到循环头，break 的边直接进入出口。循环头分别对 i 和 sum 建立 phi，continue 边保持 sum，普通边传入累加后的值。出口的 phi 对正常条件退出和 break 退出分别提供值。

```
header:
  %i = phi i32 [0, %entry], [%next, %skip], [%next, %accumulate]
  %sum = phi i32 [0, %entry], [%sum, %skip], [%added, %accumulate]
  %more = icmp slt i32 %i, %n
  br i1 %more, label %body, label %done
done:
  %result = phi i32 [%sum, %header], [%sum, %break.test]
```

LLVM verifier 能发现类型、前驱和支配关系等结构错误，但结构合法不能证明语义正确；还需运行比较。输入 n=2 时只累加 1，n=5 或 n=8 时结果均为 13，分别触发 continue 与 break。函数例子进一步使用递归阶乘和 void 输出函数，检查参数、call 和 ret 的对应关系。

数组/浮点例子将 `int a[2][3]=1,2,3,4` 表示为二维数组对象，初始化第二行剩余元素为零。数组实参传地址，按 `[3 x i32]` 的行类型通过 getelementptr 寻址，两行和为 10。浮点过程为 $\mathrm{twice}(1.5n+0.5)=3n+1$，使用 sitofp、fmul、fadd、fptosi 和 fcmp，既验证 int/float 转换，也验证 float 参数和返回值。本次只选小整数输入，使数值可精确表示；未将浮点舍入误差当作本实验结论。

（六） mem2reg 观察与结果验证

另手写一份以 alloca/load/store 保存循环变量的求和程序，使用 LLVM 自带 `opt -passes=mem2reg` 得到可提升变量的寄存器形式。转换前为 2 个 alloca、3 个 load、4 个 store、0 个 phi；转换后这些局部访存操作均消失，形成 2 个 phi。对 n=-3、0、1、5、10 分别运行，结果与原程序一致。该实验是对现成 pass 的观察，不代表本小组已自行实现编译器的 mem2reg 模块。

每个输入比较七条路径：参考程序本机/RV64、手写 IR 本机/RV64（原始和 O2），以及链接库后的 lli 运行。33 组输入产生 231 次运行比较，再对 mem2reg 结果做 5 次独立检查，共 **236/236 通过**，均检查 stdout 和退出码。

示例	运行比较数	通过数
arithmetic / short_circuit / loop_control	各 42	各 42
functions / array_float	各 35	各 35
promote_demo（含 mem2reg）	40	40
合计	236	236

表 5: 2026-10-11 实际执行结果

（七） 结论与搭档交接

实验说明手写 IR 的结构正确性、短路顺序、循环合流、数据表示和目标调用约定可以通过分层验证共同检查。优化前后相同输出是必要的正确性证据，但有限测试不能证明任意输入下等价。阶乘限定 0–12，其他输入避免 int 溢出、数组越界、除零和浮点转整数超范围；无效输入处理不在当前验证范围内。

本次 RISC-V 汇编由 llc 自动生成并经 QEMU 检验，用于验证 IR 可进入目标工具链。**这些文件不能替代搭档负责的手写汇编任务。**搭档可按本目录的 SysY 源码和预期输出编写相应汇编，再把结果接入同一验证矩阵。原先报告中的循环阶乘入门例子仍可保留；扩展例子的完整运行证据已随项目提供。

（八） 手写 RISC-V 汇编程序及验证

手写 RISC-V（RV64）汇编遵循标准调用约定（a0–a7 传参、ra 保存返回地址、s0–s11 为被调者保存寄存器）：

手写 RISC-V 汇编：fact_manual_rv.s（关键片段）

```
main:
    addi  sp, sp, -32
    sd    ra, 24(sp)
    sd    s0, 16(sp)
    sd    s1, 8(sp)
    sd    s2, 0(sp)
    call  getint          # a0 = n
    mv    s0, a0
    li    s1, 2           # i = 2
    li    s2, 1           # f = 1
.Lloop:
```

```
    bgt   s1, s0, .Ldone  # while (i <= n)
    mul   s2, s2, s1      # f = f * i
    addi  s1, s1, 1       # i = i + 1
    j     .Lloop
.Ldone:
    mv    a0, s2
    call  putint
    li    a0, 10
    call  putch
    li    a0, 0
    ld    ra, 24(sp)
    ld    s0, 16(sp)
    ld    s1, 8(sp)
    ld    s2, 0(sp)
    addi  sp, sp, 32
    ret
```

交叉编译并通过 QEMU 运行，输入 7 输出 5040，结果正确：

```
$ riscv64-linux-gnu-gcc -static fact_manual_rv.s sylib.c -o fact_manual_rv
$ echo 7 | qemu-riscv64 ./fact_manual_rv
5040
```

【待补全:更多示例程序的手写汇编与验证过程,可对比手写汇编与 `riscv64-linux-gnu-gcc -S` 自动生成汇编的差异】

（九） 共同验收状态

已有循环阶乘手写汇编完成入门验证。扩展 LLVM IR 的 236 项检查见前述章节，其中自动生成的 RISC-V 汇编不计入搭档的手写任务。更多手写汇编需由张龙飞完成后按同源程序、同输入和同预期结果接入验收。

四、 进阶要求：MLIR 多级中间表示探索

（一） 探索目标与证据边界

本部分由袁煜杰完成，研究预备工作进阶要求中的 MLIR 中端处理。首先查阅课件指定的 AscendNPU IR VecAdd 示例与方言文档，再构造上游 MLIR 的同类逐元素加法程序，真实执行多层降低、保存检查点，并验证末端运行结果。两条路线分别是官方 NPU 文档研究与本机 CPU 实验，不将后者写成前者的上板结果。

本机未发现 bishengir-compile、CANN 工具链目录或 NPU 设备节点。课件允许环境配置困难时通过文档阅读、方言差异和探索过程完成进阶说明。因此 Ascend 专有流程采用文档研究，不编造设备输出或性能数据。另从 Ubuntu 官方源下载 MLIR 21.1.8 的工具和共享库，解压到项目临时目录，不安装到系统；由脚本设置进程内的共享库搜索路径。

（二） AscendNPU IR VecAdd 的方言与层级

官方快速入门 [2] 的 VecAdd 使用 i16 元素，并区分 GM 与 UB 地址空间。输入先搬入片上缓冲，再进行向量加法，最后搬回输出；设备入口用 HACC 属性表达。编译后还需要 CANN

runtime 注册和启动设备程序。这与仅把数组指针传给 CPU 函数的执行环境不同。

架构说明 [3] 把 HFusion、HIVM、HACC 分为高层融合、硬件相关计算/搬运/同步、异构执行表达等职责。结合 HIVM 文档 [4]，可以看到片上内存和计算流水信息在特定层被显式保留。**快速入门直接给出 HIVM 级程序，不能由该输入声称已经实际观察到 HFusion 到 HIVM 的转换。**本报告的层级关系来自官方架构说明，非本机 pass 日志。

比较点	上游 CPU 实验	Ascend 官方文档路线
高层计算	Linalg 通用逐元素计算	HFusion 等较高层表达
硬件相关操作	普通 MemRef 访存	HIVM 中的计算、搬运、同步
内存空间	本机普通内存	GM、UB 等硬件空间
执行接口	MLIR 生成 C 包装函数	HACC 入口及 CANN runtime
证据	本机各层输出与运行检查	官方示例和架构文档分析

表 6: 两种路线的职责与证据差异

（三） 可执行的上游 VecAdd 设计

独立设计的 `prep/mlir/vecadd.mlir` 使用三个 `memref<16xi16>` 参数。Linalg 的三个恒等索引映射表示 $c[i] = a[i] + b[i]$，迭代属性 parallel 表明各元素计算互相独立。这保留了后续选择循环或向量化策略的空间，但本次选择的是标量循环路线。

```
linalg.generic {
  indexing_maps = [affine_map<(i)->(i)>,
                   affine_map<(i)->(i)>, affine_map<(i)->(i)>],
  iterator_types = ["parallel"]
} ins(%a, %b : memref<16xi16>, memref<16xi16>)
  outs(%c : memref<16xi16>) {
^bb0(%x: i16, %y: i16, %old: i16):
  %sum = arith.addi %x, %y : i16
  linalg.yield %sum : i16
}
```

这是本次手写的上游例子，不是复制或执行 Ascend 专有程序。输入数组设为 $a[i] = i$、$b[i] = 1$（$0 \le i < 16$），C 驱动逐项检查输出。

（四） 逐层 lowering 与检查点

上游 MLIR 文档介绍结构化 SCF 与非结构化 CF 的区分 [5]。本次根据工具可用 pass[6] 执行如下管线，每一步均保存文件：

```
mlir-opt vecadd.mlir -o 00_linalg.mlir
mlir-opt 00_linalg.mlir --convert-linalg-to-loops -o 01_scf.mlir
mlir-opt 01_scf.mlir --convert-scf-to-cf -o 02_cf.mlir
mlir-opt 02_cf.mlir --convert-to-llvm \
    --reconcile-unrealized-casts -o 03_llvm.mlir
mlir-translate --mlir-to-llvmir 03_llvm.mlir -o 04_llvm.ll
```

完整脚本 `prep/mlir/run_lowering.py` 支持系统工具或项目内便携工具。上述路径在脚本中落实为 results 下的检查点，不依赖工作目录的绝对用户名。

检查点	行数	实际观察到的操作方言
00_linalg	12	Linalg、Arith、Func
01_scf	15	SCF、MemRef、Arith、Func
02_cf	21	CF、MemRef、Arith、Func
03_llvm	67	LLVM

表 7: 实际保存的 MLIR 检查点（不把接口属性计为操作）

首先 generic 被展开为 `scf.for`，循环体出现两个 memref.load、一次 arith.addi 和一次 memref.store。接着循环被拆成比较、条件分支、循环体和回边，迭代变量通过块参数传递；结构化循环这个概念不再显式保留。再向 LLVM 方言转换时，MemRef 描述符中的已分配地址、对齐地址、偏移、长度和步长被显式展开，生成与 C 调用者衔接的包装函数。最后 `mlir-translate` 才输出真正的 LLVM IR。因此 `llvm.*` 操作所在的 MLIR 文件，与 LLVM 自身的 `.ll` 文件并不是同一种文本格式。

（五） 末端验证与分析

C 驱动构造与一维 MemRef 对应的描述符，通过 `_mlir_ciface_vecadd` 调用降低后的程序。本次 LLVM verifier 检查通过，Clang 编译链接后实际输出：

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
```

驱动逐个断言全部 16 个元素正确，退出码 0。版本、命令、stderr 和结果保存于 `prep/mlir/results/`。行数从 12 增至 67，主要由内存描述符和接口展开造成；该数字不能直接说明运行速度。本次没有测量 NPU 性能，也未执行片上搬运或同步操作。

多级 IR 的价值在于暂时保留当前优化需要的抽象，再逐步选择具体实现。高层可以讨论索引映射和融合，中层明确循环和内存行为，低层处理指针与调用约定。若直接从源程序进入 LLVM IR，这些高层信息通常需要重新推断。与此同时，降低并非仅改指令名称：类型、控制流、内存表示与接口也必须同步转换。上游工具未注册 HIVM/HACC 方言，不能简单套用本次 CPU pass 管线编译官方 NPU 输入。

五、 总结与体会

袁煜杰负责的手写 LLVM IR 与 MLIR 实验已取得可复现结果。SSA 合流、短路求值、数组布局和目标 ABI 需要同时满足结构与语义要求，多级 IR 降低则展示抽象信息如何逐层转为循环、访存与指针操作。共同报告仍需搭档补齐工作流程分析及扩展手写汇编，再统一提交。本机仅使用最小教学运行库，取得课程正式库后还需复测。

参考文献

[1] LLVM Project. LLVM Language Reference Manual. https://llvm.org/docs/LangRef.html.

[2] AscendNPU IR. 快速入门示例. https://ascendnpu-ir.gitcode.com/zh_cn/sources/introduction/quick_start/examples_zh.html.

[3] AscendNPU IR. 架构介绍. https://ascendnpu-ir.gitcode.com/zh_cn/sources/introduction/architecture_zh.html.

[4] AscendNPU IR. HIVM Dialect. https://ascendnpu-ir.gitcode.com/zh_cn/sources/developer_guide/dialects/HIVMDialect.html.

[5] MLIR Project. SCF Dialect. https://mlir.llvm.org/docs/Dialects/SCFDialect/.

[6] MLIR Project. Passes. https://mlir.llvm.org/docs/Passes/.